Proyecto Base de Datos SQL

Sistema para almacenar datos de un gimnasio

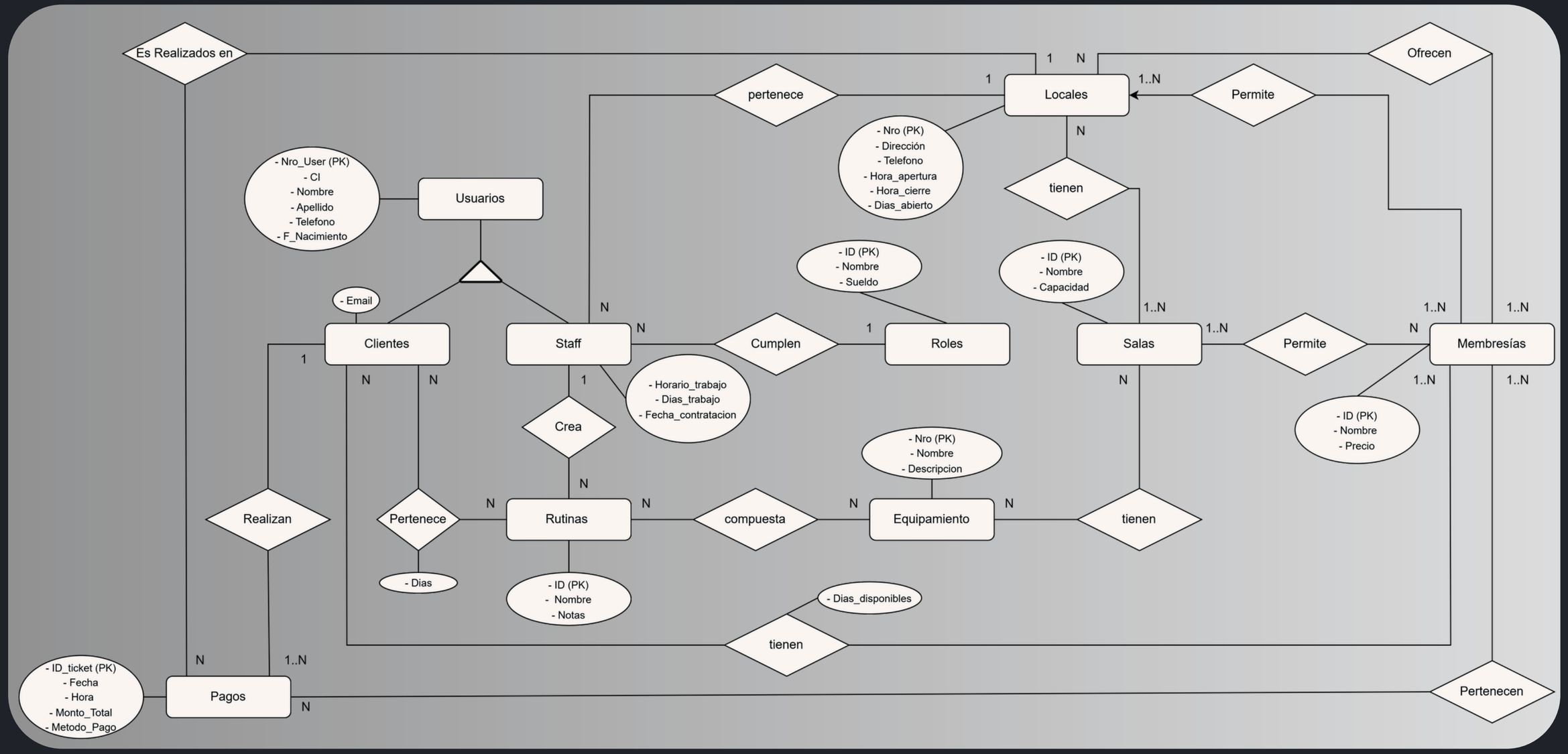
Fundamentos

Personalmente, elegí un gimnasio porque es un entorno real y dinámico que me es familiar. Me gusta el desafío de traducir procesos físicos cotidianos (como el flujo de socios, pagos y entrenamientos) en una estructura lógica, funcional y profesional.

Es fundamental implementar este tipo de negocios en una base de datos relacional porque garantiza la integridad de la información frente al error humano. Permite centralizar el control de acceso, automatizar el vencimiento de membresías, acceso de clientes a determinados servicios según sus membresías, etc.

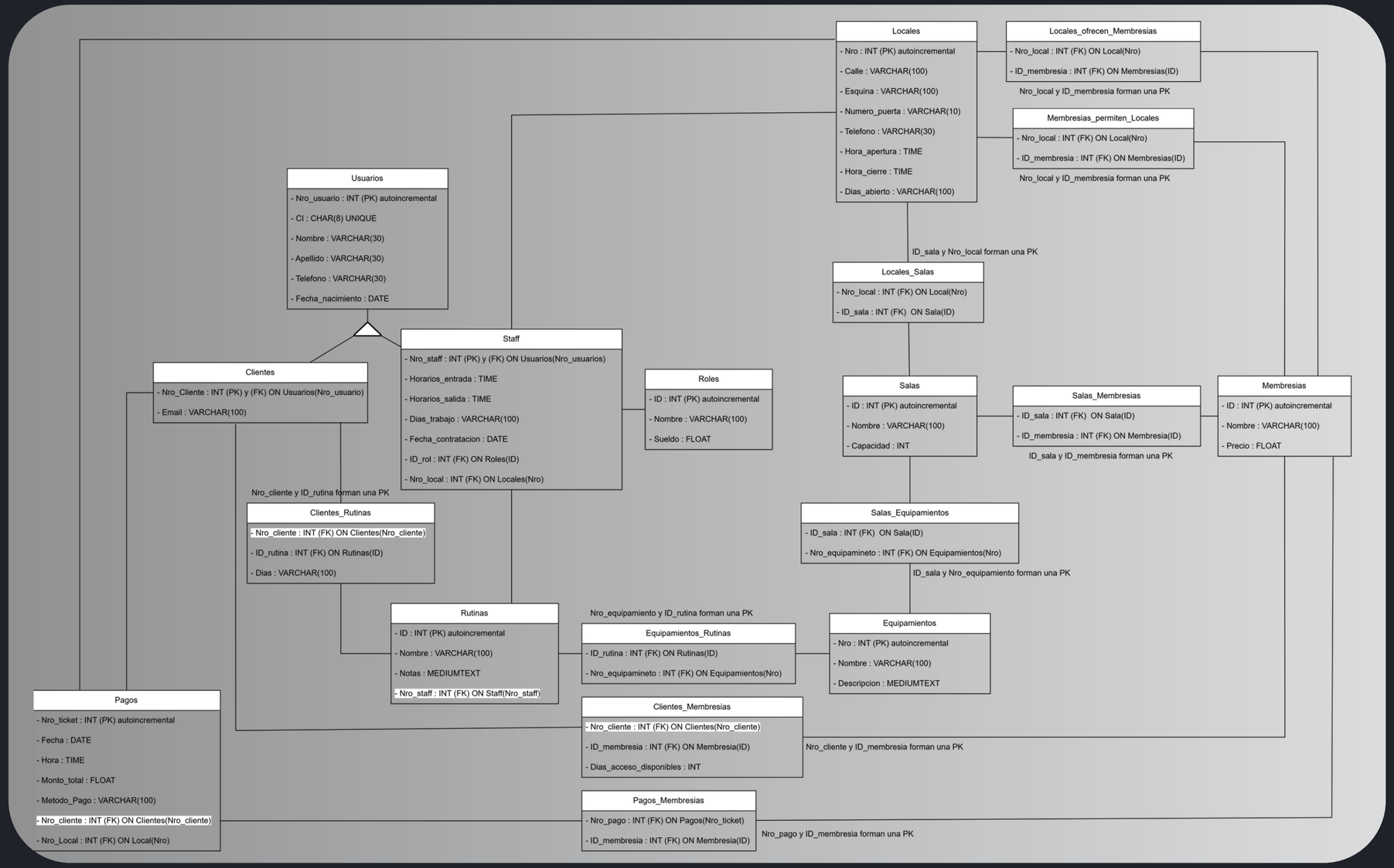
MER inicial

link al PDF



DER (sin normalizar)

link al PDF

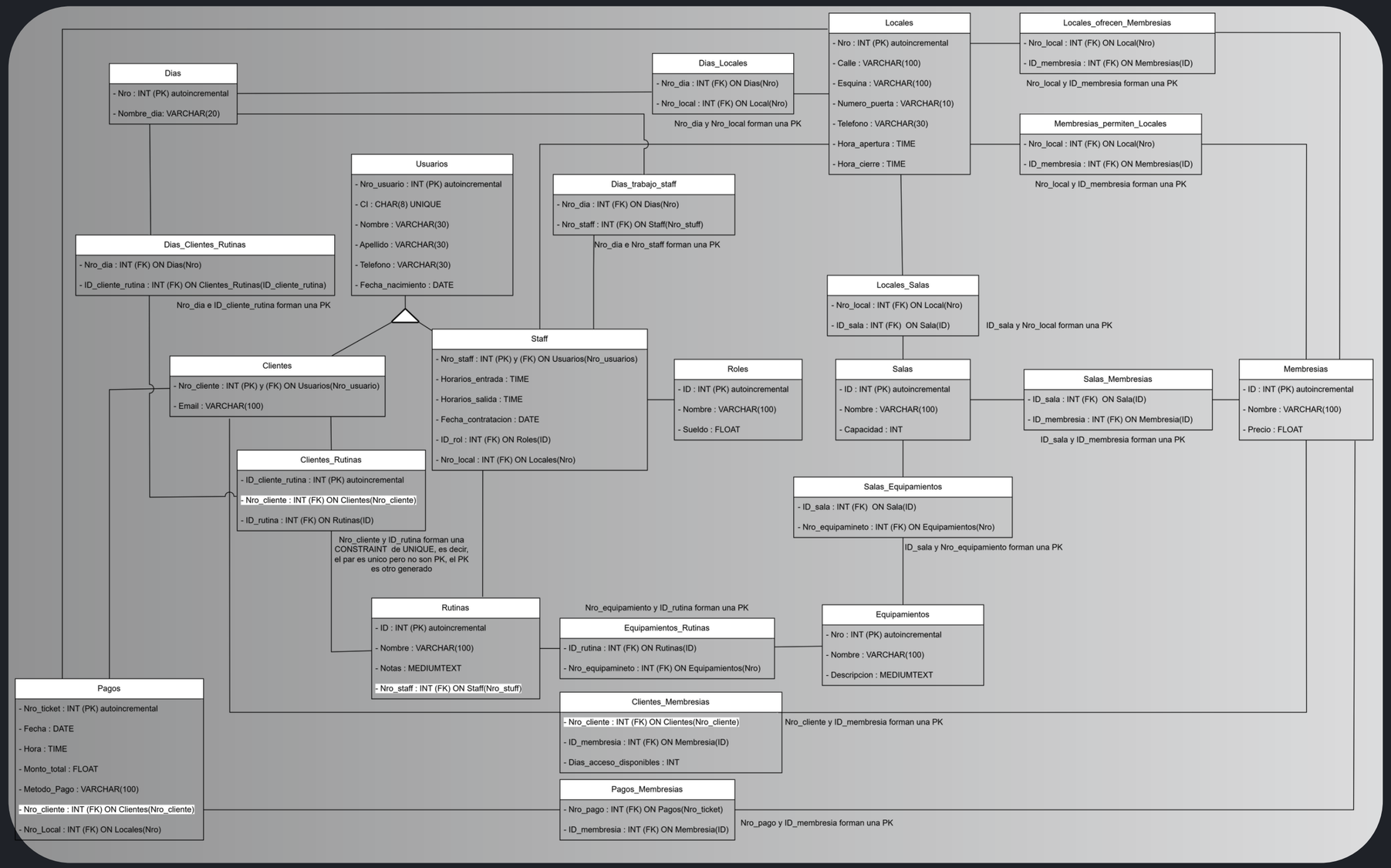


Además de las entidades principales, a causa de las relaciones con cardinalidades N:M, nacen nuevas tablas que modelan estas relaciones.

En las siguientes diapositivas se muestran las tablas en su normalización…

DER (1FN)

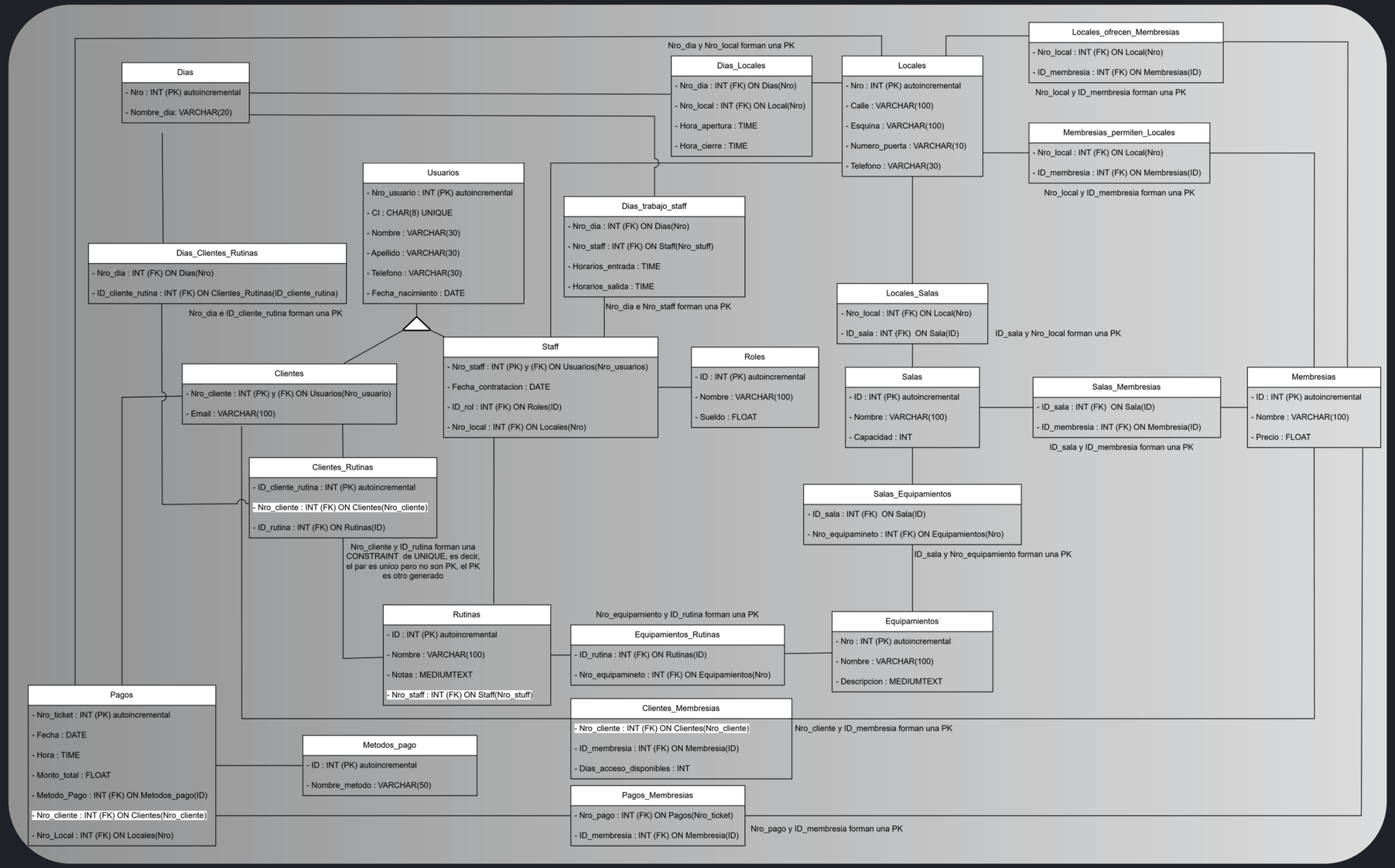
link al PDF



Para evitar que el campo días sea multivalorado, se genera una tabla ´DIAS´ con los días de la semana.
Se forman relaciones N:N con las tablas LOCAL, STAFF, y CLIENTE_RUTINA, generando a su vez las tablas resultantes de esas relaciones N:M.

DER (2FN y 3FN)

link al PDF



Se busca que no hayan campos que no dependan de la PK.

- Los campos de horarios de STAFF y LOCALES pasan a las tablas de sus relaciones.
- El método de pago se extrae en una entidad particular y se relaciona con la tabla PAGO.

Aspectos a destacar

Modelado de usuarios

Para modelar los usuarios, decidí crear una tabla con los datos generales y dos entidades separadas para Clientes y Staff, simulando así la herencia a través de claves foráneas y primarias.
Se logra que la información esté bien segmentada sin repetir datos innecesariamente.

Tercera Forma normal

Al llevar las tablas a la 2da forma normal, automáticamente también queda en 3ra forma normal ya que a niveles de este proyecto, considero que no quedan referencias transitivas.

Links a archivos

- MER inical
- DER (0FN)
- DER (1FN)
- DER (2FN y 3FN - final)
- Descripcion de tablas
- GYM.sql
- CARGA DE DATOS
- Repositorio de Github

Vistas

Agregado de vistas

Se agregaron un conjunto de vistas que consolida la información de las 24 tablas para automatizar reportes de gestión y auditoría en tiempo real. Permite monitorear la salud financiera del gimnasio comparando ingresos mensuales entre locales, calculando variaciones porcentuales y detectando tendencias de facturación sin necesidad de procesar datos manuales.

Dichas vistas fueron definidas en un archivo sql aparte el cual se puede ver en el siguiente link: Vistas

Procedimientos

Agregado de Procedimientos

Se implementaron procedimientos almacenados para centralizar la lógica de negocio y automatizar tareas operativas recurrentes. Los procedimientos incluyen validaciones críticas (como verificar la existencia de clientes o cupos disponibles) y manejan automáticamente el saldo de días de acceso, sumando o restando días según las transacciones realizadas.

Dichos procedimientos fueron definidos en un archivo SQL aparte el cual se puede ver en el siguiente link: Procedimientos

Funciones

Agregado de Funciones

Se desarrollaron funciones personalizadas para automatizar cálculos y validaciones frecuentes dentro de la base de datos. Estas herramientas permiten, por ejemplo, determinar la edad exacta de los usuarios, formatear direcciones de locales de manera uniforme o validar instantáneamente si un socio tiene permitido el ingreso a una sala y local específicos según su contrato vigente.
La principal ventaja de estas funciones es su versatilidad, ya que pueden ser llamadas directamente en reportes y consultas para transformar datos crudos en información útil (como estados de membresía "Activo" o "Vencido"). Además, incluyen manejadores de errores que aseguran que el sistema responda correctamente ante datos inexistentes.
Dichas funciones fueron definidas en un archivo SQL aparte el cual se puede ver en el siguiente link: Funciones

Triggers

Agregado de Triggers

Se implementaron triggers para automatizar la integridad de los datos y asegurar el cumplimiento de las reglas de negocio. Estos disparadores actúan como una capa de seguridad adicional que valida, por ejemplo, que ningún usuario menor de 16 años pueda ser registrado en el sistema, bloqueando la operación antes de que impacte en la base de datos.
Además, se incluyó un sistema de auditoría mediante un trigger de borrado que captura y respalda la información de los clientes eliminados en una tabla histórica. Para esto se creó una tabla CLIENTES_ELIMINADOS donde estos son almacenados.

Dichos triggers fueron definidos en un archivo SQL aparte el cual se puede ver en el siguiente link: Triggers